

IJVM Instruction Set

IJVM Instruction Set

Each instruction consists of an **opcode**, and sometimes an **operand**.

Hex	Mnemonic	Meaning
0x10	BIPUSH <i>byte</i>	Push byte onto stack
0x59	DUP	Copy top word on stack and push onto stack
0xA7	GOTO <i>offset</i>	Unconditional branch
0x60	IADD	Pop two words from stack; push their sum
0x7E	IAND	Pop two words from stack; push Boolean AND
0x99	IFEQ <i>offset</i>	Pop word from stack and branch if it is zero
0x9B	IFLT <i>offset</i>	Pop word from stack and branch if it is less than zero
0x9F	IF_ICMPEQ <i>offset</i>	Pop two words from stack; branch if equal
0x84	IINC <i>varnum const</i>	Add a constant to a local variable
0x15	ILOAD <i>varnum</i>	Push local variable onto stack
0xB6	INVOKEVIRTUAL <i>disp</i>	Invoke a method
0x80	IOR	Pop two words from stack; push Boolean OR
0xAC	IRETURN	Return from method with integer value
0x36	ISTORE <i>varnum</i>	Pop word from stack and store in local variable
0x64	ISUB	Pop two words from stack; push their difference
0x13	LDC_W <i>index</i>	Push constant from constant pool onto stack
0x00	NOP	Do nothing
0x57	POP	Delete word on top of stack
0x5F	SWAP	Swap the two top words on the stack
0xC4	WIDE	Prefix instruction; next instruction has a 16-bit index

Reading the instruction set table

The table lists every JVM opcode, its mnemonic, and what it does to the stack. A few groups worth noticing:

- **Arithmetic/logic** - `IADD` , `ISUB` , `IAND` , `IOR` all pop two words and push one result.
- **Control flow** - `GOTO` , `IFEQ` , `IFLT` , `IF_ICMPEQ` branch by changing `PC` , some conditionally based on a popped value.
- **Local variables** - `ILOAD` / `ISTORE` / `IINC` move data between the stack and the local variable frame, addressed relative to `LV` via `varnum` .
- **Stack manipulation** - `DUP` , `POP` , `SWAP` rearrange the top of the stack without touching local variables.
- **Method calls** - `INVOKEVIRTUAL` and `IRETURN` , covered next.

`BIPUSH 5 ( 0×10 0×05 )` pushes the byte value `5` onto the stack as a full word; this is how small integer literals get onto the stack in the first place.

Invoke Virtual

`INVOKEVIRTUAL` is an instruction for invoking another method.

And `IRETURN` is for exiting the method and returning to the caller.

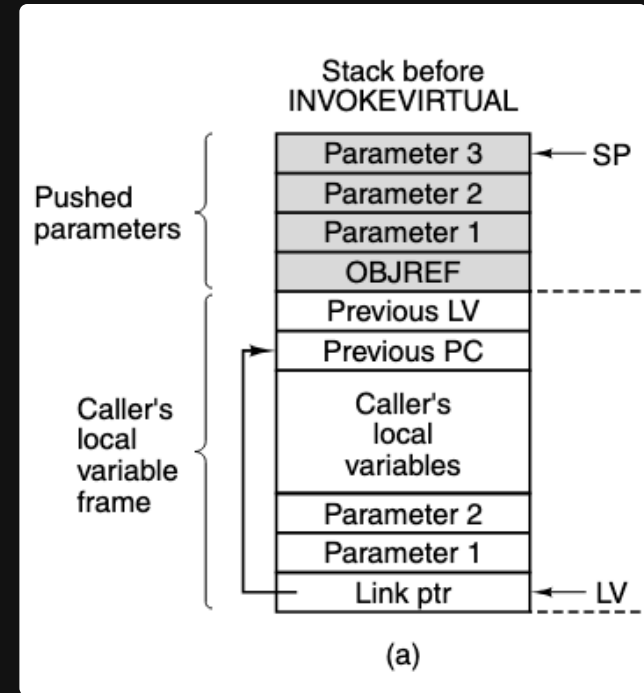
The mechanism for invoking a method is:

1. the caller pushes onto the stack a reference (pointer) to the object to be called.

This reference is a register with the memory address of the object; in the diagram this is `OBJREF`.

2. the caller pushes the method's parameters onto the stack, `Parameter 1 2 3`.

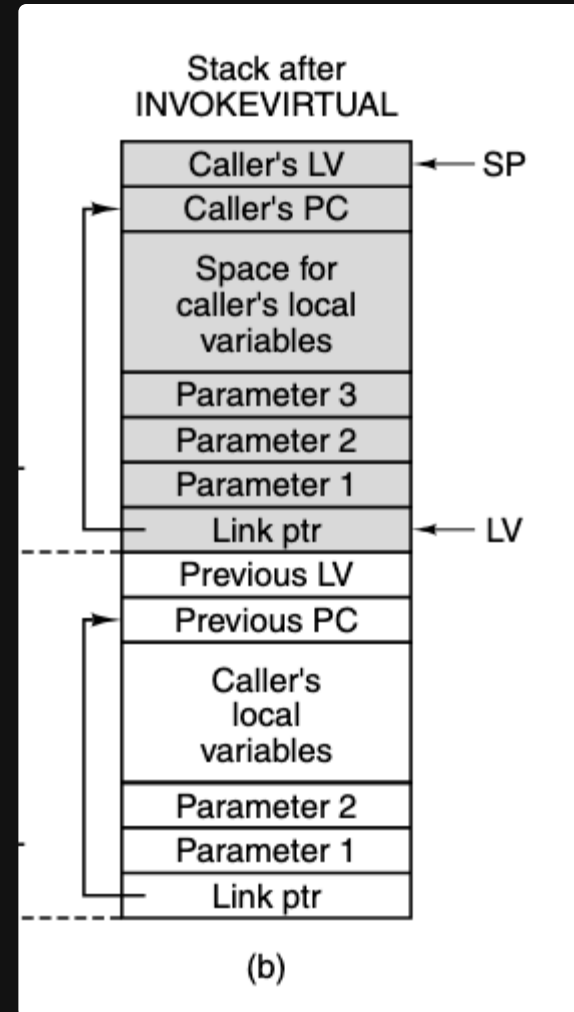
3. then `INVOKEVIRTUAL` is executed.



Invoke Virtual

The `INVOKEVIRTUAL` instruction includes a displacement which indicates the position in the constant pool that contains the start address, within the method area, for the method being invoked.

- So here you have the passed parameters
- the local variable frame for the called method is created above the parameters
- the caller's program counter (`PC`) is saved onto the stack so that when the method completes it can return to the correct location in the caller

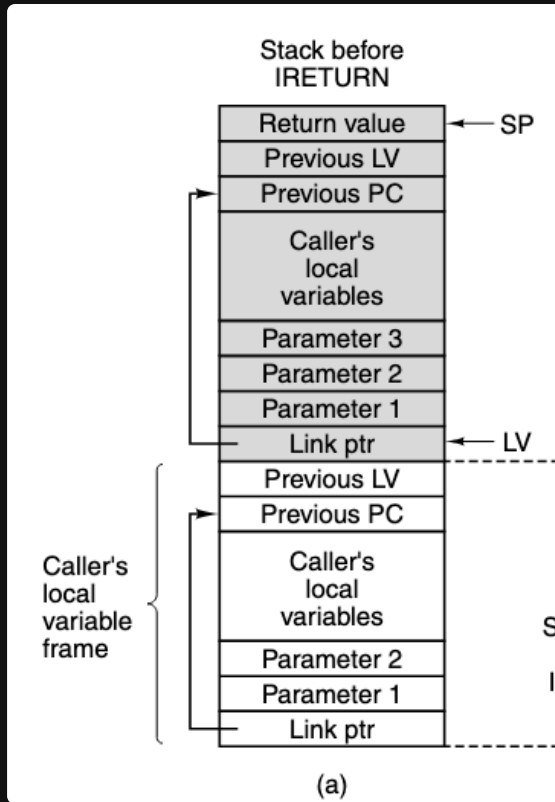


Tracing an INVOKEVIRTUAL call

Walking through the before/after diagrams together:

Step	Stack state
Before	<code>OBJREF</code> , <code>Parameter 1 2 3</code> sit on top of the caller's frame
<code>INVOKEVIRTUAL</code> executes	Caller's <code>LV</code> and <code>PC</code> are saved (pushed) so they can be restored later
New frame created	A <code>Link ptr</code> marks the base of the new frame; <code>LV</code> now points here
Space reserved	Room is reserved above the parameters for the new method's own local variables
After	<code>SP</code> points to the top of the newly reserved local variable space

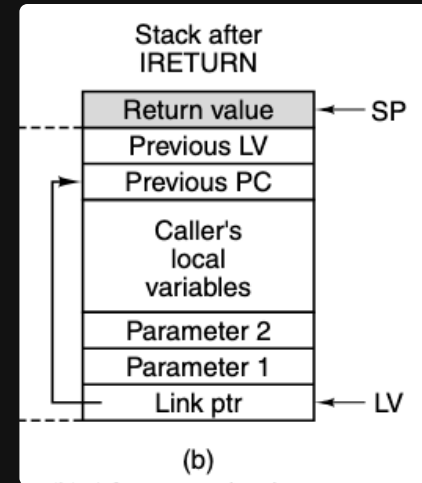
Invoke Virtual



When the method completes, an `IRETURN` instruction is executed.

It deallocates the space that was used by the returning method.

Then it restores the stack to its former state, except `OBJREF` is now replaced with the return value.



Turning Java to IJVM

Java

```
1  i = j + k;  
2  if (i == 3)  
3      k = 0;  
4  else  
5      j = j - 1;
```

IJVM

```
1  ILOAD j  
2  ILOAD k  
3  IADD  
4  ISTORE i  
5  ILOAD i  
6  BIPUSH 3  
7  IF_ICMPEQ L1  
8  ILOAD j  
9  BIPUSH 1  
10 ISUB  
11 ISTORE j  
12 GOTO L2  
13 L1: BIPUSH 0  
14     ISTORE k  
15 L2:
```

Activity: Java to JVM

using ILOAD , ISTORE , BIPUSH , IADD , ISUB , IF_ICMPEQ , GOTO

Convert this java

```
1  if (a == b)
2      x = a + 1;
3  else
4      x = b - 1;
```

Design Of The Microarchitecture Level

The Microarchitecture Level

Computers have different characteristics you need to balance: speed, cost, reliability, ease of use, energy requirements, size.

But the most important choice is:

Speed vs Cost

There are three basic approaches for increasing speed:

1. Reduce the number of clock cycles needed to execute an instruction
2. Simplify the organization so that the clock cycle time is reduced
3. Overlap the execution of instructions

Speed vs Cost

One of the ways to reduce clock cycle time is *modifying the encoding and decoding of an operation*.

The number of clock cycles needed to execute a set of operations is known as the **path length**.

We can lower the path length by adding specialized hardware

for example, adding a physical incrementer, and the price is **more hardware cost**.

Another way is to overlap the execution of instructions, known as **pipelining**.

And another, most common way, is to *create faster circuits* which are significantly more complex.

The slowest binary adder is significantly **simpler**.

While the fastest designs of the binary adder are those that use more **complex logic** to calculate carry bits in advance.

The problem with the faster designs is that they take more **real estate** compared to the simpler designs, which **directly translates to cost**.

And so a balance must be struck between speed and cost.

A concrete cost/speed tradeoff

A **ripple-carry adder** computes each bit's carry only after the previous bit's carry is ready, but the delay grows with the number of bits, since the last carry must "ripple" through every stage.

A **carry-lookahead adder** instead computes all the carries up front, in parallel, using extra logic gates for each bit position.

This is much *faster* for wide additions, but requires substantially more transistors directly increasing chip area and cost.

Adder design	Hardware complexity	Speed (for wide words)	Relative cost
Ripple-carry	Very simple, repeated unit	Slow	Low
Carry-lookahead	Complex, extra gates per bit	Fast	High

Improving performance

Every manufacturer wants their systems to run *as fast as possible*, and there are a number of techniques used to improve performance.

Roughly it falls into **two categories**:

1. implementation improvements, and
2. architectural improvements

Implementation improvements are **new ways** of building a new CPU or memory.

Modifying implementation without changing the architecture means old programs will run on the new machine, which is a **major selling point**.

Improving performance

Some kinds of improvements can be made only by changing the **architecture**.

Sometimes these are *incremental*, like adding one or two instructions, and for this reason, *software needs to change* to take advantage of the new features.

Implementation improvements are **preferred**, but the limits of architecture are constantly being hit, which requires starting from scratch again and again.

Implementation vs. architectural improvements

	Implementation improvement	Architectural improvement
Example	Faster ALU circuit, bigger cache, better pipelining	New instruction (e.g. vector/SIMD instructions)
Old software compatible?	Yes, unchanged	Only if software is updated to use it
Selling point	"Runs your existing programs faster"	"Enables new capabilities, if you adopt them"
Risk	Diminishing returns as existing designs mature	Fragmentation, some programs use the new feature and some don't

Note: This is exactly why CPU vendors keep adding *optional* instruction set extensions (like AVX on x86) rather than replacing the ISA outright, old binaries must keep running unmodified, while new software can opt in to the extension for extra performance.

Reporting

Improving performance

1. Cache Memory (p239)
2. Branch Prediction (p335)
3. Out of Order Execution (p340)
4. Speculative Execution (p345)

Instruction Set Architectures

Instruction Set Architectures

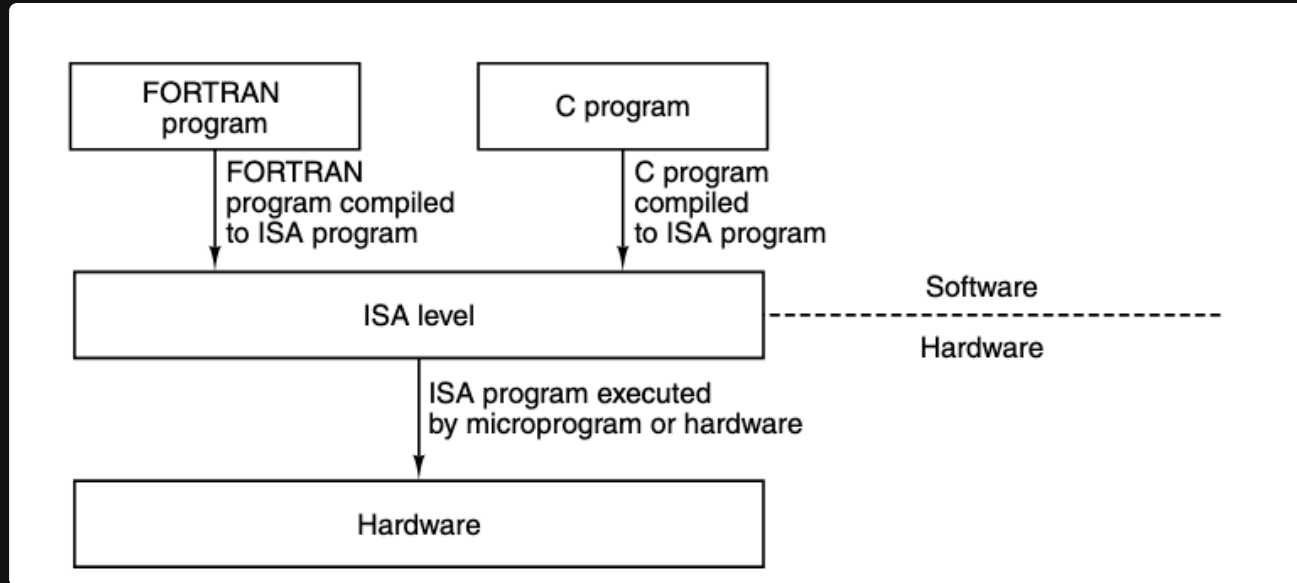
This is the interface between the software and the hardware. And while it might be possible to have the hardware directly execute programs written in C or Java or some high-level language, this would **not** be a good idea.

1. Performance gained from compiling would be lost
2. Computers would need only be able to run programs written in one language
3. The cost of hardware would be significantly higher

And so, to let computers run programs written in different languages, we need an intermediate language, known as **machine language** or **machine code**.

All code written in high-level languages is compiled or interpreted into machine code before it is executed by the computer.

Instruction Set Architectures



The ISA as a stable contract

Both the FORTRAN program and the C program are compiled down to the *same* ISA-level representation, even though they started as different source languages.

Below that line, the hardware (or a microprogram, as with the Mic-1/IJVM) only ever has to deal with one format: ISA instructions.

This is precisely why the same compiled program can, *in principle*, run unmodified on any hardware implementation that honors the same ISA

the ISA acts as a **stable contract** that *decouples* compiler writers from chip designers.

Instruction Set Architectures

Ideally, when a new machine is being designed, the architects, the engineers, the hardware designers, and the software developers would work together to create a system with **good performance, low cost, and ease of programming**.

In reality, when a new machine comes along, the first question is:

"Is it compatible with its predecessor?"

And the second question is:

"Can I run my old operating system on it?"

And the third question is:

"Will it run all my existing application programs unmodified?"

And if any of these are answered no, the designer will need to make a very strong case for why the new machine is worth the cost of switching.

This is why ISAs tend to be the same between models, or at least backwards compatible with older models.

This makes them unable to take advantage of new features of the hardware or the architecture when running old programs.

Properties of the ISA Level

The ISA level is defined by how the machine appears to a machine-language programmer, or more commonly, the compiler writer.

Some architectures are specified by a formal defining document, often produced by an industry consortium, like the ARM architecture.

These documents have things such as:

1. **normative** sections, which impose requirements, and
2. **informative** sections, which are intended to help the reader but are not part of the formal definition

In the normative section, phrases such as *shall*, *may not*, and *should* are used to indicate requirements.

Executing a reserved opcode shall cause a trap

A well-written ISA reference is essential for anyone writing compilers, assemblers, debuggers, or other system software.

Normative vs. informative, in practice

In an ARM architecture reference manual, a sentence like "Executing a reserved opcode **shall** cause a trap" is *normative* every conforming implementation **must** behave this way, or it isn't a correct implementation of the architecture.

By contrast, a sentence like "This design choice makes it easier to detect programming errors" would be *informative*

it explains the *reasoning*, but doesn't itself impose a requirement; two conforming implementations could disagree on such informative commentary without either being "wrong."

This distinction matters enormously for tool builders: a compiler or assembler author needs to know exactly which behaviors are *guaranteed* (normative) versus merely *typical* or *motivational* (informative), since only the former can be relied upon across all conforming hardware.

Assembly Language Level

Assembly Language Level

Programs that convert a user's program written in a high-level language into machine code are called translators.

With **source language** and **target language**, where each language is defined at a level. Since if a processor can execute both source and target language, there's no need for translation.

So translation is used when a processor is available for the target language but not the source language.

If done correctly, running the target and source language should have the **exact** same effect.

This also means that it's possible to implement new **levels for which there is no processor** by first translating programs written for that level to a target level.

Translation and Interpretation

In **translation**, the original program in the source language is not directly executed. Instead, a new program is made, usually called an **object program** or **executable binary program**, whose execution is carried out only **after** the translation is complete.

In translation there are two steps:

1. Generation
2. Execution

In **interpretation**, only one step exists, which is the execution of the source program. No equivalent program is generated first.

And while it's being executed, each instruction is read, *interpreted*, and executed one at a time.

Compilers

Translators can be roughly divided into two categories depending on the relation between the *source* and *target* languages.

When the source language is a symbolic representation for a numerical machine language (`IADD = 0x60`), it's called an **assembler**.

When the source language is a high-level language like C, and the target is *either* a numerical machine language *or* a symbolic representation of a machine language, it's called a **compiler**.

Assembly -> **Machine Code** = Assembler

C -> **Machine Code** or **Java** -> **assembly** = Compiler

Assembly

A pure assembly language is a language in which each statement produces exactly one machine instruction, a **one-to-one** correspondence.

It's important to note that when talking about *ISA*, it is physically in **binary or hex**.

Assembly language is a **symbolic representation** of the ISA, which is easier for humans to read and write.

Its main point of difference from high-level languages is that assembly has access to **all** the machine's features, which high-level languages often hide or abstract away.

And that it can only ever run on one family of machine which it was written for.

Why assembly

Assembly is difficult. It takes longer to write, to debug, and to maintain.

So why would anyone ever want to use assembly directly?

In today's world, the answer is that **you almost never would.**

Compiler writers will make systems that will out-perform any assembly program you would ever write. And unless you plan on working on extremely low-level systems, you will never need to write assembly.

If you do learn and use assembly effectively, it allows you two main advantages:

1. Performance - you can write code that is faster and smaller, which is critical for systems such as embedded systems
2. Access - you can access low-level system features that are not available in high-level languages, such as directly accessing hardware

Assembly in the age of AI

Modern AI coding assistants are trained overwhelmingly on high-level languages (Python, JavaScript, Java, C, etc.), simply because that's what the vast majority of publicly available code is written in.

Assembly is comparatively rare, highly architecture-specific, and full of context that isn't in the code itself (calling conventions, register usage rules, memory layout assumptions)

This makes it one of the areas where AI-generated code is **least reliable**.

Note: this doesn't make assembly *less* important to understand. Verifying, debugging, and reasoning about AI-generated low-level or performance-critical code still requires a human who actually understands what the instructions are doing, and why a compiler (or an AI) chose to emit them that way.

So learning assembly isn't just "good for learning" in the abstract, it's increasingly one of the few areas where human expertise can't be shortcut by simply asking an AI to write the code for you.